# CO4 – ASSIGNMENT 4

C Programming: Structures, Unions and Storage Classes

| | |
|---|---|
| **Student Name** | B. Janani |
| **Register Number** | 1912260492 |
| **Assignment** | CO4 Assignment 4 |
| **Submission Document** | Solutions, source code and execution evidence |

Prepared from the six tasks in the supplied CO4 Assignment 4 question paper. Each implemented program was compiled and executed with GCC using C11 mode; the terminal output captures are included with the corresponding source code.

# Submission contents

• Task 1 – Patient register data design and repair

• Task 2 – Courier parcel tracking with nested structures

• Task 3 – Weather sensor packets using a tagged union

• Task 4 – Structure passing, pointers and account interest

• Task 5 – Automatic, static, global/extern and register storage classes

• Task 6 – Canteen inventory and billing design challenge

• Appendix – Build and execution notes

**Compiler:** GCC with `-std=c11 -Wall -Wextra`. Output addresses in Task 4 vary between executions and systems.

**Important:** The output images in this report are terminal captures generated from the compiled programs in this working environment. The exact memory addresses and the inactive-union-member value are implementation-dependent.

# TASK 1 – DATA DESIGN REVIEW

## 1.1 Diagnose the original design

**Parallel arrays:** The same patient is spread across several arrays. If one array is sorted or updated without all others, a name can be paired with another patient's age, phone number or medical data.

**Phone field char[10]:** A C string needs a terminating '\0'. Ten digits need at least 11 bytes; char[10] can cause truncation or an out-of-bounds string read.

**Blood pressure in one float:** Blood pressure has two separate values (systolic and diastolic). One float cannot represent them reliably as two independently accessible measurements.

**Date stored as char[8]:** The format is unclear and difficult to validate or compare. A structured day/month/year representation supports checking and sorting.

**No capacity check:** When count reaches 100, the next write goes beyond every array and can corrupt memory.

**Global variables:** Global arrays and count can be changed by any function, making errors harder to locate and testing/reuse more difficult.

**Sorting by age:** Parallel arrays require every related array to be swapped in exactly the same way. Missing one swap breaks patient-record integrity.

## 1.2 Repaired structure design and addPatient()

```
typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;
typedef struct {
    char name[50];
    int age;
    char phone[16];
    BloodPressure bp;
    Date visitDate;
} Patient;

#define MAX_PATIENTS 100
int addPatient(Patient patients[], int *count, const Patient *newPatient) {
    if (*count >= MAX_PATIENTS) return 0;
    patients[*count] = *newPatient;
    ++(*count);
    return 1;
}
```

The array stores complete Patient records. The function receives the new patient through a const structure pointer and receives count by pointer so it can safely update the caller's count.

## 1.3 Design defence (under 100 words)

A structure groups all fields belonging to one patient, so a complete record can be copied or swapped as one unit. Sorting by age moves the whole Patient and keeps the fields together. Functions can receive a pointer to one record instead of many separate arguments, avoiding unnecessary copies. Nested Date and BloodPressure types make the meaning of each value clear and allow those types to be reused. New fields can be added to Patient without maintaining several parallel arrays.

## Alternative concept and complexity

Parallel arrays can be useful when processing one field at a time, when data is column-oriented, or when a program needs compact access to one homogeneous field. With an array of structures, a swap moves one whole Patient (approximately sizeof(Patient) bytes, including any padding). With parallel arrays, a swap must exchange each field in its own array; the total bytes moved are approximately the sum of the field widths. Both selection sort versions take $O(n^2)$ comparisons and $O(1)$ extra swap space. The array-of-structures form is usually safer for record integrity; exact byte counts depend on the compiler and alignment.

## 1.4 Complete program and output

```
#include <stdio.h>
#include <string.h>

typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;

typedef struct {
```

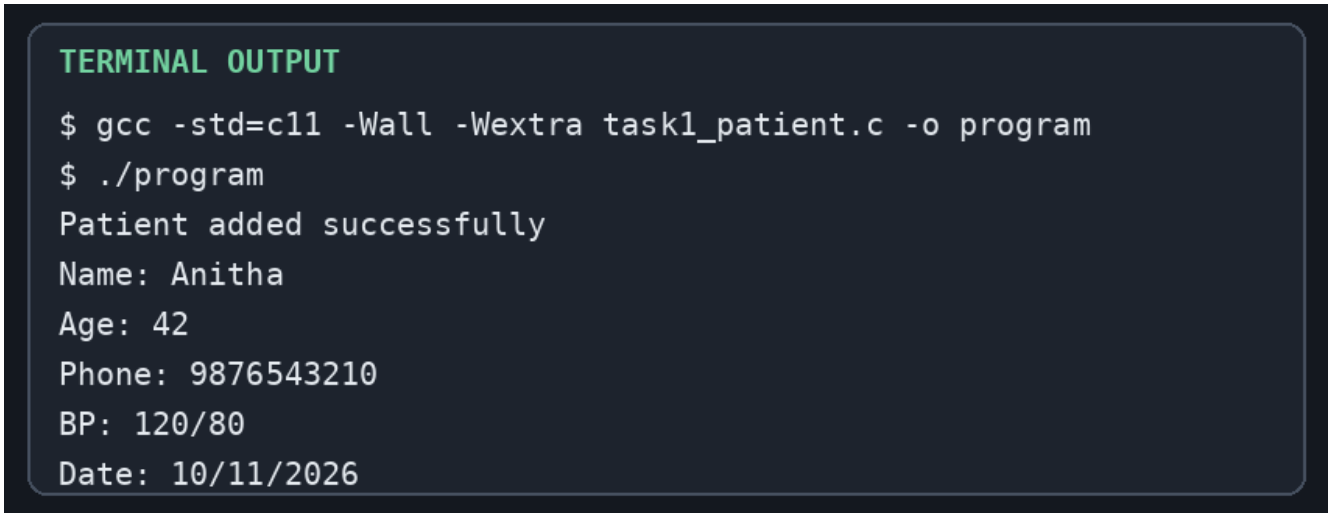
```
    char name[50];
    int age;
    char phone[16];       /* room for 10 digits plus terminator */
    BloodPressure bp;
    Date visitDate;
} Patient;

#define MAX_PATIENTS 100

int addPatient(Patient patients[], int *count, const Patient *newPatient) {
    if (*count >= MAX_PATIENTS) return 0;
    patients[*count] = *newPatient;
    ++(*count);
    return 1;
}

int main(void) {
    Patient patients[MAX_PATIENTS] = {0};
    int count = 0;
    Patient p = {"Anitha", 42, "9876543210", {120, 80}, {10, 11, 2026}};

    if (addPatient(patients, &count, &p)) {
        printf("Patient added successfully\n");
        printf("Name: %s\nAge: %d\nPhone: %s\nBP: %d/%d\nDate: %02d/%02d/%04d\n",
               patients[0].name, patients[0].age, patients[0].phone,
               patients[0].bp.systolic, patients[0].bp.diastolic,
               patients[0].visitDate.day, patients[0].visitDate.month,
               patients[0].visitDate.year);
    } else {
        puts("Register is full");
    }
    return 0;
}
```



# TASK 2 – COURIER PARCEL TRACKING

## 2.1 Entities and nesting

**Parcel** is the main entity. It contains parcel ID, weight, booking date, sender, receiver, service type and charge. **Address** contains name (where applicable), city and PIN code. **Date** contains day, month and year. Sender and receiver each contain an Address; Parcel contains Date and both Address values.

## 2.2 Structure definitions

```
typedef struct { int day, month, year; } Date;
typedef struct { char name[40]; char city[20]; int pincode; } Address;
typedef enum { NORMAL, EXPRESS } ServiceType;
typedef struct {
    int id;
    float weight;
    Date booked;
    Address sender;
    Address receiver;
    ServiceType service;
    float charge;
} Parcel;
Parcel bookings[100];
```

## 2.3 Operation pseudocode

**Sort by weight (ascending):** For each position i from 0 to n–2, find the index of the lightest parcel from i through n–1. If the minimum index differs from i, swap the complete Parcel structures.

**List by receiver city:** Read the target city. Visit each parcel from 0 through n–1. If receiver.city matches the target city, print that parcel's ID, receiver details and weight.

## 2.4 Access paths

Third parcel receiver PIN using array and dot: `bookings[2].receiver.pincode`. Using a pointer to the third parcel: `ptr->receiver.pincode`, where `ptr = &bookings[2]`.

## Completed TODO program

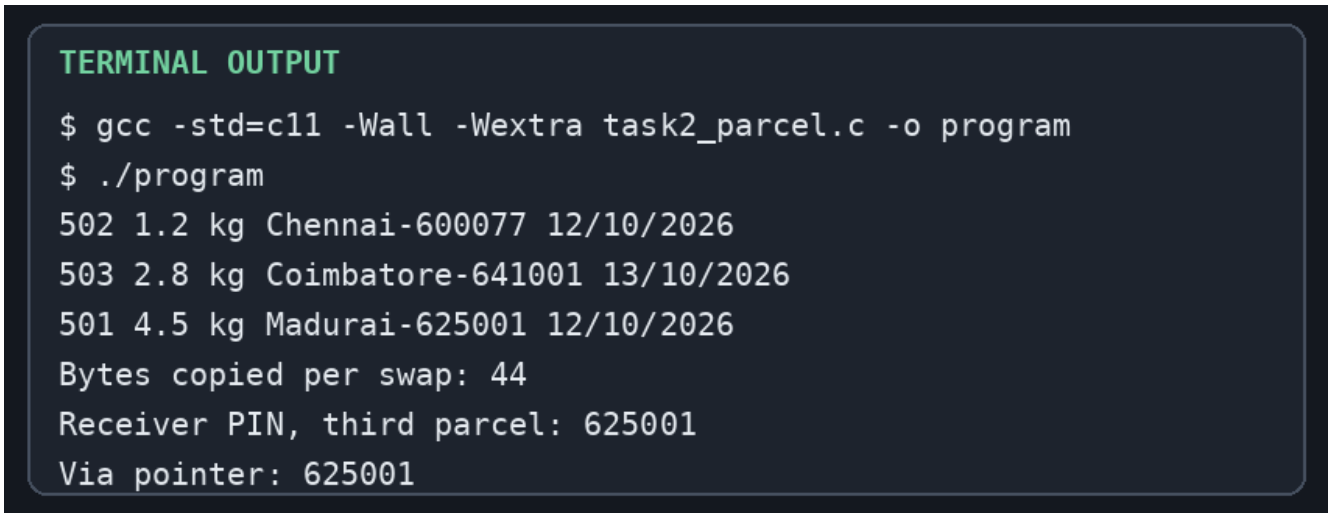
```
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%06d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min = i;
        for (int j = i + 1; j < n; ++j)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) {
            struct parcel temp = p[i];
            p[i] = p[min];
            p[min] = temp;
        }
    }
}

int main(void) {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; ++i) print_parcel(&list[i]);
    printf("Bytes copied per swap: %zu\n", sizeof(struct parcel));
    printf("Receiver PIN, third parcel: %d\n", list[2].to.pincode);
    const struct parcel *ptr = &list[2];
    printf("Via pointer: %d\n", ptr->to.pincode);
    return 0;
}
```




## Complexity and flat-structure alternative

Selection sort performs O(n²) comparisons. Each actual swap copies one complete structure; this build reports sizeof(struct parcel) = 44 bytes, so the temporary plus two assignments move approximately three structure-sized copies at source level. A flat Parcel could hold fields such as receiverCity, receiverPincode, day, month and year directly. Its access paths become shorter (for example, p->receiverPincode), but Address and Date cannot be reused as cleanly. Nested and flat designs may have the same total member storage; sizeof is the final authority because padding and alignment are implementation-dependent.

# TASK 3 – UNIONS AND MEMORY LAYOUT

## 3.1 Structure versus tagged union

A structure containing float celsius, int percent and char status[8] reserves storage for all three values at the same time. A tagged union stores a tag plus one shared storage area; only the member identified by the tag is valid. Since each sensor packet contains exactly one value and memory is limited, the tagged union is the better fit.

## 3.2 Size prediction

On the tested typical 64-bit GCC environment, enum kind occupies 4 bytes and the union occupies 8 bytes because its largest member is status[8]. The union's alignment requirement is 4 bytes. The tag takes 4 bytes and the union begins at offset 4; total structure size is 12 bytes. A structure with all fields would typically use 4 + 4 + 8 = 16 bytes, subject to padding.

## 3.3 Completed tagged-union program

```
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};

void show(const struct reading *r) {
    switch (r->type) {
        case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
        case HUMIDITY:    printf("Humidity %d %%\n", r->value.percent); break;
        case STATUS:      printf("Status %s\n", r->value.status); break;
        default:          puts("Unknown packet"); break;
    }
}

int main(void) {
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; ++i) show(&log[i]);

    /* Misuse demonstration: do not interpret this as a real humidity value. */
    struct reading wrong = {TEMPERATURE, {.celsius = 31.5f}};
    printf("Misuse: humidity member read after temperature write = %d (implementation-dependent)\n",
           wrong.value.percent);
    return 0;
}
```

```
TERMINAL OUTPUT
$ gcc -std=c11 -Wall -Wextra task3_union.c -o program
$ ./program
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
Misuse: humidity member read after temperature write = 1107034112 (implementation-dependent)
```

## 3.4 Misuse experiment and explanation

The program writes 31.5 into celsius and then reads percent. The observed integer in this GCC run is shown in the output capture, but it is not a meaningful humidity reading. Reading a different union member does not convert the temperature into a valid percentage; the result is implementation-dependent and should not be used by the application. Always check the tag before reading the union.

## Alternative and complexity

Three separate arrays avoid a tag per element but require identifying the packet type and maintaining matching indices across arrays. Reading a known-type array entry is O(1), but mixed packet order is harder to preserve. A void-pointer design with separately allocated values requires allocation metadata, casts and explicit type tracking; access can be O(1), but each allocation adds allocator overhead and may fragment memory. A tagged union provides O(1) tag check and access, contiguous storage and no per-packet heap allocation. Its space per packet is sizeof(struct reading), 12 bytes in this tested build.

# TASK 4 – STRUCTURE FORENSICS

## 4.1 Defects

• **Balances unchanged:** addInterest receives struct account by value, so it modifies only a local copy. The caller's list element is unchanged.

• **Compilation error:** p is a pointer. Use p->number and p->balance, not p.number and p.balance.

## 4.2 Address evidence and 4.3 repair

The first address printed in main and the address received by the first addInterest call are equal, because the repaired function receives &list[i]. The program changes the original object through the pointer.

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account *a, float rate) {
    printf("Inside function address: %p\n", (void *)a);
    a->balance += a->balance * rate / 100.0f;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;
    printf("In main address:          %p\n", (void *)&list[0]);
    for (int i = 0; i < 3; ++i) addInterest(&list[i], 4.0f);
    puts("Updated balances:");
    for (int i = 0; i < 3; ++i)
        printf("%d %.2f\n", p[i].number, p[i].balance);
    printf("First account via arrow: %d %.2f\n", p->number, p->balance);
    return 0;
}
```




## 4.4 Passing guidelines

**By value:** use when a small structure should be copied and the function must not modify the caller's object. **By pointer:** use when the function must update the original object or avoid copying a large structure. **By const pointer:** use to read a structure without copying it and prevent accidental modification through that pointer.

## Alternative return-by-value design

```
struct account addInterestCopy(struct account a, float rate) {
    a.balance += a.balance * rate / 100.0f;
    return a;
}
/* caller */
for (int i = 0; i < 3; ++i)
    list[i] = addInterestCopy(list[i], 4.0f);
```

The return-by-value version copies a structure into the function parameter and returns a modified structure; the assignment stores the returned value. The pointer version passes one address and updates the original directly. Both take O(1) time per account, but the value version can move sizeof(struct account) bytes for parameter/return operations, while pointer passing transfers only an address. Compiler optimizations may remove some copies. The pointer version generally uses less temporary stack space.

# TASK 5 – STORAGE CLASSES

## 5.1 Four implementations

**Automatic local:** a new local token is initialized to 1000 on every call, so every call returns 1001. **Static local:** initialized once and retains its value, producing 1001, 1002, 1003. **Global with extern:** a file-scope variable can be defined in one source file and declared with extern in another. **Register:** a loop variable may be requested to reside in a CPU register; modern compilers decide the actual placement.

## Two-file global/extern example

```
/* counter.c */
int shared_counter = 1000;
int next_global_token(void) { return ++shared_counter; }

/* main.c */
#include <stdio.h>
extern int shared_counter;
int next_global_token(void);
int main(void) {
    printf("%d %d %d\n", next_global_token(),
           next_global_token(), next_global_token());
    return 0;
}
/* Build: gcc -std=c11 counter.c main.c -o tokens */
```

The executable included in this report also demonstrates the global counter in one file so that all examples can be compiled and run together.

## Completed TODO program and output

```
#include <stdio.h>

int shared_counter = 1000; /* global counter, shared across source files */
int next_global_token(void) { return ++shared_counter; }

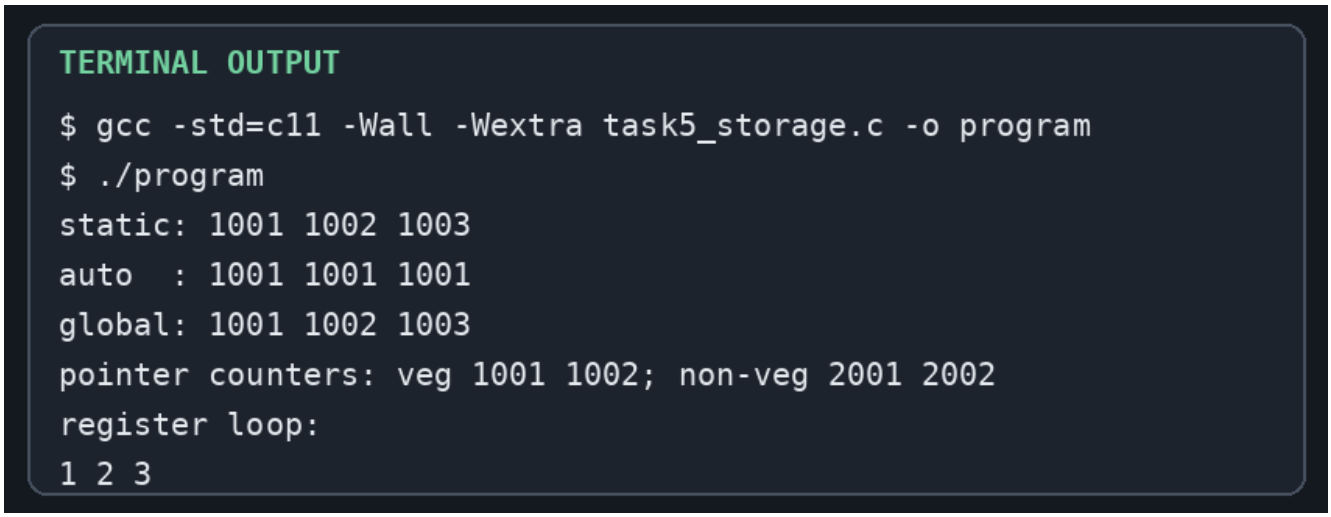
int next_token(void) {
    static int token = 1000; /* initialized once; retains value */
    return ++token;
}

int auto_token(void) {
    int token = 1000;        /* recreated on every call */
    return ++token;
}

int main(void) {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    int g1 = next_global_token(), g2 = next_global_token(), g3 = next_global_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto  : %d %d %d\n", a1, a2, a3);
    printf("global: %d %d %d\n", g1, g2, g3);

    int veg = 1000, nonveg = 2000;
    int v1 = ++veg, v2 = ++veg, n1 = ++nonveg, n2 = ++nonveg;
    printf("pointer counters: veg %d %d; non-veg %d %d\n", v1, v2, n1, n2);

    puts("register loop:");
    for (register int i = 1; i <= 3; ++i) printf("%d ", i);
    putchar('\n');
    return 0;
}
```




## 5.2 Two independent counters

With static locals, create separate functions or separate counter objects for veg and non-veg; separate functions are needed if the state is hidden inside each function. With a global design, define two global variables (for example veg_counter and nonveg_counter) and declare both with extern where needed. With caller-owned counters, use int next_token(int *counter) { return ++(*counter); }; call it with &veg and &nonveg. The caller-owned pointer design needs the least change to support any number of independent counters.

## 5.3 Comparison

| Storage | Scope | Lifetime | Initial value | Main risk |
|---|---|---|---|---|
| auto | Block | While block executes | Indeterminate unless initialized | Value does not persist |
| static local | Block | Entire program | Zero by default / explicit initializer once | Hidden shared state across calls |
| global / extern | File scope (extern declaration in other files) | Entire program | Zero by default / explicit initializer | Any visible function may modify it |
| register | Block | While block executes | Indeterminate unless initialized | Cannot take its address |

C does not allow the address-of operator on a register object because the register storage-class specifier requests that the object be accessed as a register-like value and the implementation is not required to provide an addressable memory location.

Function arguments are not evaluated in a specified left-to-right order in C. Therefore, calling next_token() three times as arguments of one printf can display the returned values in a different order, even though the counter is incremented three times. Use separate statements when a particular order is required.

All counter operations are O(1). A static or global counter occupies storage for the entire program. A caller-owned counter occupies the caller's storage and can be passed to any function; two independent counters are naturally supported by passing two different addresses.

# TASK 6 – COLLEGE CANTEEN INVENTORY AND BILLING

## 6.1 Entities and testable requirements

• Item: ID, name, price, stock, revenue and item kind.

• Packaged item: expiry date. Cooked item: preparation date/time. Combo: up to three component item IDs and quantities.

• Sale: item ID, quantity and calculated amount.

• A sale is accepted only when quantity is positive and does not exceed available stock.

• Accepted sales reduce stock and increase revenue by price × quantity.

• Packaged items expiring today or within the next two days are listed.

• The daily summary is sorted by revenue in descending order.

## 6.2 Proposed data model

Use a Date structure for calendar dates, an enum tag for the three item kinds, a Combo structure for up to three component references, and a tagged union inside Item. Store the day's items in an array of structures. Sales can be recorded through a pointer to the selected Item.

## 6.3 Critical design decisions

Stock must be checked before it is reduced; reject invalid quantities and insufficient stock. Expiry comparisons should use validated calendar dates, not strings. This demonstration uses a same-month/year day comparison to keep the sample short; a production program should convert dates to a day number or use a proper date library so month/year boundaries and leap years are handled. No global or static state is required for the inventory: pass the item array and count to functions. A static/global transaction counter is justified only if a persistent shared sequence is a real requirement.

## 6.4 Structure and function design

```
typedef struct { int day, month, year; } Date;
enum item_kind { PACKAGED, COOKED, COMBO };
typedef struct { int ids[3], quantities[3], count; } Combo;
typedef struct {
    enum item_kind kind;
    int id;
    char name[24];
    float price;
    int stock;
    union { Date expiry; Date prepared; Combo combo; } detail;
    float revenue;
} Item;

int record_sale(Item *item, int quantity);
void show_expiry_warnings(const Item items[], int n, Date today);
void sort_by_revenue(Item items[], int n);
```

**Sale pseudocode:** Find the item by ID. Reject if quantity is not positive or quantity exceeds stock. Otherwise subtract quantity from stock, add price × quantity to revenue, and print confirmation.

**Expiry pseudocode:** For each item, check whether its kind is PACKAGED. Compare today's date with its expiry date. If expiry is from today through two days ahead, print a warning.

## 6.5 Working implementation and test evidence

```
#include <stdio.h>
#include <string.h>

typedef struct { int day, month, year; } Date;
enum item_kind { PACKAGED, COOKED, COMBO };

typedef struct {
    int ids[3];
    int quantities[3];
    int count;
} Combo;

typedef struct {
    enum item_kind kind;
    int id;
```

```
    char name[24];
    float price;
    int stock;
    union {
        Date expiry;        /* PACKAGED */
        Date prepared;      /* COOKED: date prepared */
        Combo combo;        /* COMBO: component item IDs and quantities */
    } detail;
    float revenue;
} Item;

int record_sale(Item *item, int quantity) {
    if (quantity <= 0) return 0;
    if (quantity > item->stock) return 0;
    item->stock -= quantity;
    item->revenue += item->price * quantity;
    return 1;
}

int expires_within_two_days(Date today, Date expiry) {
    /* Simple day-number comparison for dates in the same month/year (demo data). */
    if (today.year != expiry.year || today.month != expiry.month) return 0;
    int diff = expiry.day - today.day;
    return diff >= 0 && diff <= 2;
}

void show_expiry_warnings(const Item items[], int n, Date today) {
    puts("Expiry warnings:");
    for (int i = 0; i < n; ++i) {
        if (items[i].kind == PACKAGED &&
            expires_within_two_days(today, items[i].detail.expiry))
            printf("%s expires %02d/%02d/%04d\n", items[i].name,
                   items[i].detail.expiry.day, items[i].detail.expiry.month,
                   items[i].detail.expiry.year);
    }
}

void sort_by_revenue(Item items[], int n) {
    for (int i = 0; i < n - 1; ++i)
        for (int j = i + 1; j < n; ++j)
            if (items[j].revenue > items[i].revenue) {
                Item t = items[i]; items[i] = items[j]; items[j] = t;
            }
}

int main(void) {
    Item items[3] = {
        {PACKAGED, 1, "Juice", 25.0f, 5, {.expiry = {12, 11, 2026}}, 0},
        {COOKED, 2, "Sandwich", 40.0f, 4, {.prepared = {10, 11, 2026}}, 0},
        {COMBO, 3, "Meal Combo", 70.0f, 3,
         {.combo = {{1, 2, 0}, {1, 1, 0}, 2}}, 0}
    };
    Date today = {10, 11, 2026};

    printf("Sale Juice x2: %s\n", record_sale(&items[0], 2) ? "OK" : "REJECTED");
    printf("Sale Sandwich x8: %s\n", record_sale(&items[1], 8) ? "OK" : "REJECTED");
    printf("Sale Meal Combo x1: %s\n", record_sale(&items[2], 1) ? "OK" : "REJECTED");
    show_expiry_warnings(items, 3, today);

    sort_by_revenue(items, 3);
    puts("Revenue summary (highest first):");
    for (int i = 0; i < 3; ++i)
        printf("%s revenue=%.2f stock=%d\n",
               items[i].name, items[i].revenue, items[i].stock);
    return 0;
}
```

```
TERMINAL OUTPUT
$ gcc -std=c11 -Wall -Wextra task6_canteen.c -o program
$ ./program
Sale Juice x2: OK
Sale Sandwich x8: REJECTED
Sale Meal Combo x1: OK
Expiry warnings:
Juice expires 12/11/2026
Revenue summary (highest first):
Meal Combo revenue=70.00 stock=2
Juice revenue=50.00 stock=3
Sandwich revenue=0.00 stock=4
```

Test evidence: Juice quantity 2 is accepted; Sandwich quantity 8 is rejected because stock is 4; one Meal Combo is accepted; Juice is reported as expiring within two days; the revenue summary is sorted highest-first.

## Alternative: linked list versus array

A linked list can insert a new item at the head in O(1) time and remove a known node in O(1) time when its previous node is already known. Searching by ID is O(n); locating a node to remove is also O(n). An array provides O(1) indexed access and good cache locality, but insertion/removal in the middle may require shifting O(n) elements. Each linked-list node uses extra memory for one or more next pointers and may incur allocation overhead. For a small, fixed daily inventory, an array is simple and efficient; a linked list is useful when the collection changes size frequently.

# APPENDIX – BUILD AND EXECUTION

The six source files are provided in this report and were separately compiled and executed. To reproduce the runs, save each source listing using its filename and run:

```
gcc -std=c11 -Wall -Wextra task1_patient.c -o task1_patient\n./task1_patient\n\nRepeat with task2_parcel.c through task6_canteen.c.
```

The output screenshots are placed directly below each corresponding source listing in this PDF. The C programs are sample demonstrations using fixed test data; they do not collect real patient, parcel, bank, sensor or canteen records.

**Submission reminder:** This PDF is prepared locally. It has not been uploaded to GitHub or the SIMATS portal. Those uploads require the student's authenticated accounts and repository/portal details.